Language

Автор сайта

Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Dagger 2
сделайте код чище
курс от автора сайта
StartAndroid
MADE WITH MYBANNERMAKER.COM

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 086 подписчиков

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 15. Навигация и ViewModel

05 февраля 2023

В этом уроке мы рассмотрим важные особенности работы ViewModel при использовании навигации

Про ViewModel мы уже говорили, про навигацию - тоже. Теперь можно обсудить, как они работают вместе. Тут есть о чем поговорить, потому что ViewModel работает по разному с Navigation и без.

Чтобы увидеть эту разницу, рассмотрим пример с двумя экранами: UserListScreen и UserScreen. На первом экране отображается список пользователей. Нажимаем на пользователя - открывается второй экран с детальной информацией. Также на втором экране мы будем получать ViewModel и делать интересные выводы.

Чтобы не городить кучу кода, пример будет максимально простым. На UserListScreen вместо списка у нас будет просто два текста-кнопки: User 1 и User 2. А на UserScreen в качестве детальной информации о пользователе будем отображать только ID.

Экран UserListScreen:

```
@Composable
fun UserListScreen(
    onUser1Click: () -> Unit,
    onUser2Click: () -> Unit,
) {
    Column {
        Text(text = "Users screen")
        Text(
            text = "User 1",
            modifier = Modifier.clickable(onClick = onUser1Click)
        )
        Text(
            text = "User 2",
            modifier = Modifier.clickable(onClick = onUser2Click)
        )
    }
}
```

Тексты в Column, по нажатию вызываем лямбды.

Экран UserScreen:

```
@Composable
fun UserScreen(
    id: String?,
    userViewModel: UserViewModel = viewModel()
) {
    Text(text = "User $id")
}
```

На вход принимает id и отображает его в тексте. UserViewModel - это пустая ViewModel, которая ничего не делает и никак не используется.

Мы будем открывать экран UserScreen с разными пользователями и проверять, получает он один и тот же экземпляр UserViewModel или разные.

Чтобы это проверить, добавим в UserScreen логирование:

```
@Composable
fun UserScreen(
    id: String?,
    userViewModel: UserViewModel = viewModel()
) {
    Text(text = "User $id")
    Log.d(TAG,"user $id")
    Log.d(TAG,"viewModel ${userViewModel.hashCode().toHexString()}")
}
```

Будем выводить в лог ID пользователя и хэшкод модели.

В качестве самодельной навигации используем то же решение, что и на прошлом уроке:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Column(modifier = Modifier.fillMaxSize()) {
                var route by remember { mutableStateOf("userList") }

                Box(modifier = Modifier.weight(1f)) {
                    when (route) {
                        "userList" -> UserListScreen(
                            onUser1Click = { route = "user/1" },
                            onUser2Click = { route = "user/2" }
                        )
                        "user/1" -> UserScreen("1")
                        "user/2" -> UserScreen("2")
                    }
                }

                Text(
                    text = "Users",
                    modifier = Modifier.clickable { route = "userList" }
                )
            }
        }
    }
}
```

Чтобы выполнять навигацию, мы меняем значение переменной route. Это приводит к тому, что when отображает нужный нам экран.

Запускаем приложение и поочередно открываем экран UserScreen для User 1 и User 2

```
Лог:
user 1
viewModel 34c1a18

user 2
viewModel 34c1a18

user 1
viewModel 34c1a18
```

Каждый раз мы получаем один и тот же экземпляр UserViewModel. И это не очень удобно.

В не Compose приложениях мы привыкли к тому, что открывая новый экран, мы получаем от системы новый экземпляр модели. Так происходит потому, что Activity и фрагмент являются ViewModelStoreOwner. Они сами создают и держат модели.

Почему с Compose экранами это не работает? Потому что экраны в Compose - это просто Composable функции. Они не являются ViewModelStoreOwner и не могут сами управлять моделями.

Откуда тогда в Composable экран приходят модели? Давайте разбираться.

Для получения модели мы используем функцию viewModel(). Если открыть ее исходники, мы там увидим параметр *viewModelStoreOwner: ViewModelStoreOwner*. И т.к. мы туда ничего не передаем, то используется значение по умолчанию:

```
viewModelStoreOwner: ViewModelStoreOwner = checkNotNull(LocalViewModelStoreOwner.current)
```

LocalViewModelStoreOwner.current - именно этот объект и отвечает за предоставление моделей нашим Composable экранам.

Чтобы узнать, что за объект там скрывается, давайте его достанем и выведем в лог его класс:

```
@Composable
fun UserScreen(
    id: String?,
    userViewModel: UserViewModel = viewModel()
) {
    Text(text = "User $id")
    Log.d(TAG,"user $id")
    Log.d(TAG,"viewModel ${userViewModel.hashCode().toHexString()}")
    Log.d(TAG,"viewModelStoreOwner ${LocalViewModelStoreOwner.current?.javaClass?.simpleName}")
}
```

Результат:

```
user 1
viewModel 243c420
viewModelStoreOwner MainActivity

user 2
viewModel 243c420
viewModelStoreOwner MainActivity

user 1
viewModel 243c420
viewModelStoreOwner MainActivity
```

Таинственным объектом оказалось MainActivity.

Т.к. Composable функции запускаются в Activity, то оно и является для них держателем ViewModel. А значит, время жизни моделей определяется временем жизни Activity, а не Composable экранов. Пока не закроем Activity, оно будет держать экземпляры моделей.

Получается, что когда мы открыли UserScreen первый раз, в Activity создался экземпляр UserViewModel. И этот экземпляр оно дает нам во все последующие открытия UserScreen. Потому что с точки зрения Activity, никакие экраны не менялись. Мы как были в этом Activity, так и продолжаем в нем находиться.

Использование Navigation Compose решает эту проблему.

Заменим в примере нашу навигацию на Navigation:

```
Column(modifier = Modifier.fillMaxSize()) {
    val navController = rememberNavController()

    NavHost(
        navController = navController,
        startDestination = "userList",
        modifier = Modifier.weight(1f)
    ) {
        composable("userList") { UserListScreen(
            onUser1Click = { navController.navigate("user/1") },
            onUser2Click = { navController.navigate("user/2") }
        ) }
        composable(
            route = "user/{id}",
            arguments = listOf(navArgument("id") { type = NavType.StringType })
        ) {
            val userId = it.arguments?.getString("id")
            UserScreen(userId)
        }
    }

    Text(
        text = "Users",
        modifier = Modifier.clickable { navController.navigate("userList") }
    )
}
```

Код нам знаком по прошлому уроку. Добавляем в NavHost экраны, используем navController для навигации и указываем аргумент для UserScreen.

Запускаем этот вариант, открываем разные UserScreen и смотрим лог:

```
user 1
viewModel 47dcf0f
viewModelStoreOwner NavBackStackEntry

user 2
viewModel d6f5203
viewModelStoreOwner NavBackStackEntry

user 1
viewModel 37d34b7
viewModelStoreOwner NavBackStackEntry
```

Каждый UserScreen экран получает свой экземпляр модели UserViewModel. Это работает благодаря тому, что держателем моделей теперь является NavBackStackEntry, а не Activity.

NavBackStackEntry - это сущность навигации. Когда мы с помощью NavController открываем новый экран, для этого экрана создается NavBackStackEntry и помещается в BackStack. Для каждого экрана эта сущность своя. И она же для него является ViewModelStoreOwner. Поэтому каждый экран получает свои экземпляры моделей из своего хранилища. А когда экран закрывается, то NavBackStackEntry убирается из BackStack, а все ее модели уничтожаются.

Т.е. при использовании навигации, каждому экрану предоставляется свое хранилище моделей, которое привязано к времени жизни этого экрана.

Shared ViewModel

Иногда бывает необходимость получать один экземпляр модели на нескольких экранах. Рассмотрим, как это можно сделать в Navigation Compose.

Мы уже выяснили, что в функции viewModel() есть параметр viewModelStoreOwner, где мы можем указать, кто будет отвечать за создание и хранение моделей. По умолчанию при использовании навигации там используется NavBackStackEntry текущего экрана. Это ограничивает модель временем жизни этого экрана. Но мы можем использовать этот параметр, чтобы указать нужный нам viewModelStoreOwner и, тем самым, продлить время жизни модели.

Рассмотрим тот же пример, что и ранее, когда мы из UserListScreen открываем UserScreen. Но теперь на этих экранах нам надо получать один и тот же экземпляр общей модели UsersSharedViewModel.

Чтобы максимально упростить код, уберем userId. Вместо User 1 и User 2, будет просто User.

UserListScreen:

```
@Composable
fun UserListScreen(
    onUserClick: () -> Unit,
    usersSharedViewModel: UsersSharedViewModel = viewModel()
) {
    Column {
        Text(text = "Users screen")
        Text(
            text = "User",
            modifier = Modifier.clickable(onClick = onUserClick)
        )
    }
}
```

Получаем модель UsersSharedViewModel и по нажатию на текст User вызываем лямбду.

UserScreen:

```
@Composable
fun UserScreen(
    usersSharedViewModel: UsersSharedViewModel = viewModel()
) {
    Text(text = "User")
}
```

Тоже получаем модель UsersSharedViewModel

Навигация:

```
NavHost(
    navController = navController,
    startDestination = "userList",
    modifier = Modifier.weight(1f)
) {
    composable("userList") {
        UserListScreen(
            onUserClick = { navController.navigate("user") },
        )
    }
    composable("user") { UserScreen() }
}
```

Что будет происходить при таком сценарии?

Когда мы открываем экран UserListScreen, то для него создается сущность NavBackStackEntry. Назовем ее userListEntry.

Далее мы из UserListScreen открываем экран UserScreen. Для него создается своя сущность NavBackStackEntry. Назовем ее userEntry.

Т.е. у нас сформировался такой BackStack из экранов и их сущностей:
UserListScreen (userListEntry) > UserScreen (userEntry)

userListEntry создала свой экземпляр UsersSharedViewModel для UserListScreen, а userEntry - свой, для UserScreen. Из-за этого у нас два разных экземпляра модели.

Но мы можем схитрить и сделать так, что UserScreen будет просить модель не у своей сущности userEntry, а у userListEntry. Тогда он получит тот же экземпляр модели, что и UserListScreen. Для этого надо найти сущность userListEntry в стэке.

Это можно сделать с помощью метода getBackStackEntry, указав соответствующий route:

```
val userListEntry = navController.getBackStackEntry("userList")
```

Перепишем код, где мы добавляем экран UserScreen в NavHost:

```
NavHost(...) {
    ...
    composable("user") { backStackEntry ->
        val userListEntry = navController.getBackStackEntry("userList")
        UserScreen(
            usersSharedViewModel = viewModel(userListEntry)
        )
    }
    ...
}
```

Теперь мы не просто вызываем UserScreen, а используем userListEntry в функции viewModel(), чтобы получить экземпляр модели с экрана UserListScreen и передать его в UserScreen.

Студия сообщает нам, что вызов метода getBackStackEntry надо обернуть в remember с ключом backStackEntry, это сделает процесс получения userListEntry более эффективным:

```
composable("user") { backStackEntry ->
    val userListEntry = remember(backStackEntry) {
        navController.getBackStackEntry("userList")
    }
    UserScreen(
        usersSharedViewModel = viewModel(userListEntry)
    )
}
```

Полный код в NavHost выглядит так:

```
NavHost(
    navController = navController,
    startDestination = "userList",
    modifier = Modifier.weight(1f)
) {
    composable("userList") {
        UserListScreen(
            onUserClick = { navController.navigate("user") },
        )
    }
    composable("user") { backStackEntry ->
        val userListEntry = remember(backStackEntry) {
            navController.getBackStackEntry("userList")
        }
        UserScreen(
            usersSharedViewModel = viewModel(userListEntry)
        )
    }
}
```

Теперь UserListScreen и UserScreen получают один экземпляр UsersSharedViewModel из userListEntry. Экран UserListScreen делает это автоматически, по умолчанию. А экрану UserScreen мы немного помогаем.

Время жизни этого экземпляра модели совпадает с временем жизни экрана UserListScreen.

Activity

Есть и более глобальный вариант получения общей модели. В качестве ViewModelStoreOwner мы можем использовать не сущность из стэка, а Activity.

Получить его можно так:

```
val activity = LocalViewModelStoreOwner.current
```

Но тут важно, где этот код будет вызван. Когда мы вызывали его в Composable экране с навигацией, мы получали NavBackStackEntry. А нам надо получить Activity, поэтому код надо вызвать на уровне Activity:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            ...
            val activity = checkNotNull(LocalViewModelStoreOwner.current)
            NavHost(
                navController = navController,
                startDestination = "userList",
                modifier = Modifier.weight(1f)
            ) {
                composable("userList") {
                    UserListScreen(
                        onUserClick = { navController.navigate("user") },
                        usersSharedViewModel = viewModel(activity)
                    )
                }
                composable("user") {
                    UserScreen(
                        usersSharedViewModel = viewModel(activity)
                    )
                }
            }
            ...
        }
    }
}
```

Мы вызываем LocalViewModelStoreOwner.current не внутри экранов NavHost, а в коде, который выполняется в Activity. Поэтому и получаем в результате не NavBackStackEntry из навигации, а Activity.

Используем это Activity для получения модели UsersSharedViewModel в экранах UserListScreen и UserScreen. Время жизни этой модели будет совпадать с временем жизни Activity.

hiltViewModel

В уроке про ViewModel мы рассматривали вариант получения модели при использовании Hilt. И там все было просто.

Есть модель с не пустым конструктором, т.е. она на вход требует какие-то объекты:

```
@HiltViewModel
class UserViewModel @Inject constructor(
    private val repository: SomeRepository
): ViewModel() { … }
```

Есть экран, который получает эту модель:

```
@Composable
fun UserScreen(
    userViewModel: UserViewModel = viewModel ()
) { … }
```

Без навигации все это нормально работало. Hilt успешно создавал создавал модель с репозиторием и предоставлял ее нам.

Но если мы попробуем сделать то же самое с подключенной навигацией, то получим ошибку:

java.lang.RuntimeException: Cannot create an instance of class ru.startandroid.mycomposeapplication.UserViewModel

Причина все та же - сменился ViewModelStoreOwner.

Когда мы не использовали навигацию, то в качестве ViewModelStoreOwner выступало Activity. А у него была настроена интеграция с Hilt. Поэтому оно умело собирать все необходимые объекты для создания модели.

С навигацией же за создание модели отвечает NavBackStackEntry. У этой сущности нет интеграции с Hilt. Она не знает как создать репозиторий, который необходим для модели. Она может создать только модель с пустым конструктором.

Чтобы сделать NavBackStackEntry умнее, есть функция hiltViewModel(). Под капотом она работает так же, как и viewModel(), но при этом выполняет одно важное дополнительное действие. Она из Activity (которое интегрировано с Hilt) вытаскивает фабрику моделей и передает ее в NavBackStackEntry. Эта фабрика умеет использовать Hilt для создания моделей, требующих другие объекты. В итоге, под капотом hiltViewModel() мы получаем прокачанный NavBackStackEntry, который знает про Hilt и умеет им пользоваться для создания моделей с не пустым конструктором.

Чтобы использовать эту функцию, необходимо добавить зависимость:

```
implementation 'androidx.hilt:hilt-navigation-compose:1.0.0'
```

И заменить viewModel() на hiltViewModel()

```
@Composable
fun UserScreen(
    userViewModel: UserViewModel = hiltViewModel()
) { … }
```

savedStateHandle

Еще одна полезная особенность навигации и моделей - это savedStateHandle.

Если Composable экран использует ViewModel:

```
@Composable
fun UserScreen(
    id: String?,
    userViewModel: UserViewModel = hiltViewModel()
) { … }
```

и у этого экрана есть аргументы, которые передаются с помощью route:

```
composable(
    route = "user/{id}",
    arguments = listOf(navArgument("id") { type = NavType.StringType })
) {
    val userId = it.arguments?.getString("id")
    UserScreen(userId)
}
```

то значения аргументов можно получить сразу в модели с помощью SavedStateHandle:

```
class UserViewModel @Inject constructor(
    private val savedState: SavedStateHandle,
): ViewModel() {

    init {
        val userId = savedState.get<String>("id")
        …
    }

}
```

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Комментарии

# **Опечатки в коде** — **Евгений** 12.02.2023 16:22
В разделе sharedViewModel после введения переменной userListEntry следует 2 блока кода (отрывки из NavHost) и в обоих мы в функцию view Model передаем некий параметр usersEntry, вместо userListEntry, а в блоке с полным NavHost уже параметр у viewModel именно userListEntry
Ответить

# **RE: Опечатки в коде** — **Dmitry Vinogradov** 20.02.2023 02:17
Пофиксил, спасибо!
Ответить

# **SavedStateHandle без Hilt** — **Евгений** 24.04.2023 11:57
Подскажите, а получение SavedStateHandle во ViewModel работает только с Hilt и функцией hiltViewModel(), или есть способы без?
Ответить

# **RE: SavedStateHandle без Hilt** — **Dmitry Vinogradov** 28.04.2023 17:01
Без Hilt тоже должно работать, с обычной функцией by viewModels()

https://developer.android.com/topic/libraries/architecture/viewmodel/viewmodel-savedstate#setup
Ответить

# **Передача объектов** — **Azat** 18.10.2023 07:06
Что бы вы использовали, если бы вам нужно было передавать объекты в другой экран? Нужен профессиональный совет)
Ответить

# **RE: Передача объектов** — **Dmitry Vinogradov** 02.11.2023 11:13
Общую ViewModel (на уровне Activity). На базе этого можно создать какой-то механизм под свои нужды. Аргументы навигации для этого не предназначены.
Ответить

# **remember(backStackEntry)** — **Георгий** 18.10.2023 17:45
"Студия сообщает нам, что вызов метода getBackStackEntry надо обернуть в remember с ключом backStackEntry, это сделает процесс получения userListEntry более эффективным".
А почему так, подскажите пожалуйста? Как часто и когда меняется backStackEntry когда мы переходим на UserScreen?
Ответить

# **RE: remember(backStackEntry)** — **Dmitry Vinogradov** 02.11.2023 11:19
Не могу представить случай, когда backStackEntry мог бы поменяться. Выглядит как перестраховка. Или может быть подготовка к каким то будущим изменениям.
Ответить

# **RE: remember(backStackEntry)** — **Pavlo** 06.01.2024 20:13
Когда разбирал пример

вот этот код
Text(
text = "Users",
modifier = Modifier.clickable { navController.navigate("userList") }
)

меняет backStackEntry, когда вы нажимаете назад - он остается тем же, а если кликаете на этот текст, он меняется (ну у меня менялся) и как следствие меняется viewModelStoreOwner и usersSharedViewModel будет другая.

Но если заменить на navController.popBackStack(route = "userList", inclusive = false)
То будет норм :)
Ответить

Обновить список комментариев

RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.